Prof. Dr. Stefan Leutenegger
Teaching Assistants: Shi Chen, Cédric Le Gentil, Alex Brandes, and Yannick Burkhardt

Exercise 8: Keypoint Matching & Bundle Adjustment

Session date: 28/04/2026

Task 1: Keypoint Matching (Python)

Before implementing the matching logic, it is essential to understand the tools:

- **SIFT:** A feature detector that finds points of interest that are robust to changes in scale and rotation.
- **Keypoint:** A distinct pixel location, such as a corner, that serves as a reliable landmark because it can be uniquely identified across different images of the same scene.
- **Descriptor:** A 128-dimensional vector that describes the local gradient information around a keypoint. We compare these vectors to find correspondences between different views of the same scene.

Given two sets of SIFT keypoint descriptors, D_1 and D_2, your task is to implement the following functions to find the best matches while minimizing outliers. After each subtask, display the matches with the provided `drawMatches` function for a reprojection error check (based on ground truth poses). It will plot correct matches in green and outliers in red.

i) To find similar features, we calculate the Euclidean distance between descriptors. Since nested `for`-loops are inefficient in Python, use vectorization and the the following expansion of the L2 norm:

$$\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2(\mathbf{a}\cdot\mathbf{b}) \tag{1}$$

In `E8_keypoint_matching.py`, implement `calculateDescriptorDistances(des1, des2)`. It should return a distance matrix of size $N \times M$, where N and M are the number of descriptors in image 1 and 2, respectively. Use `np.dot` or the `@` operator for the matrix product.

ii) Unfortunately, this simple approach results in many false matches, requiring further refinement. A match (i, j) is considered *mutual* if descriptor j is the nearest neighbor to i in image 2, **and** descriptor i is the nearest neighbor to j in image 1. Implement `getMutualMatchingMask(dist, best_index)` to do this cross-checking.

- `dist`: The $N \times M$ distance matrix from Subtask 1.
- `best_index`: An array of indices where `best_index[i]` is the index of the closest descriptor in image 2 for the i-th descriptor in image 1.
- Return a boolean mask where `True` indicates mutual matches.

This improves the matching. However, the ratio of correct and false matches is still too low.

iii) Many false matches occur because a descriptor is similarly close to multiple different points. In this paper, David Lowe proposed the "Ratio Test": a match is kept only if the distance to the *closest* neighbor is significantly smaller than the distance to the *second closest* neighbor.

$$\frac{d_{\text{1st\_nearest}}}{d_{\text{2nd\_nearest}}} < \text{threshold} \tag{2}$$

Implement `getRatioMask(dist, sorted_indices, threshold=0.7)`.

- `sorted_indices`: A matrix containing indices of descriptors sorted by distance (from `np.argsort`).
- Return a boolean mask where `True` indicates the match passes the ratio test.

The remaining outliers are now few enough to be robustly handled during Bundle Adjustment, where the optimization process can smoothly reconcile these small errors to achieve a consistent 3D reconstruction.

Task 2: Bundle Adjustment (Python)

Here, you will implement a small Bundle Adjustment problem to find camera poses that are consistent with their observations. Consider the world coordinate frame $\underrightarrow{\mathcal{F}}_W$ and the i^{th} camera coordinates $\underrightarrow{\mathcal{F}}_{C_i}$. To be found are all N camera poses $\mathbf{p}_i := [{}_W\mathbf{t}_{C_i}, \mathbf{q}_{WC_i}]^T$ and landmarks ${}_W\mathbf{l}_j$ minimising the cost

$$c = \frac{1}{2}\sum_{i=1}^{N}\sum_{j\in\mathcal{J}(i)} \mathbf{e}_{ij}^T\mathbf{e}_{ij},$$

with the reprojection error

$$\mathbf{e}_{ij} = \tilde{\mathbf{u}}_{ij} - \mathbf{u}(\mathbf{R}_{C_iW}\,{}_W\mathbf{l}_j - \mathbf{R}_{C_iW}\,{}_W\mathbf{t}_{C_i}),$$

and the set $\mathcal{J}(i)$ of landmarks visible in the i^{th} camera. Data associations between keypoint measurements and landmarks, $\tilde{\mathbf{u}}_{ij}$ are already given. for $\mathbf{u}(\cdot)$, use a pinhole projection model without distortion, camera centre c_1, c_2 and equal focal lengths f. Furthermore, the local minimal parameterisation $\boldsymbol{\alpha}$ should be used on the orientations $\mathbf{q}_{WC_i}$.

i) Write out the reprojection error with the projection model, and provide the Jacobians $\frac{\partial \mathbf{e}_{ij}}{\partial {}_W\mathbf{t}_{C_i}}$, $\frac{\partial \mathbf{e}_{ij}}{\partial \boldsymbol{\alpha}_{WC_i}}$, and $\frac{\partial \mathbf{e}_{ij}}{\partial {}_W\mathbf{l}_j}$.

ii) In `E8_bundle_adjustment.py`, the camera poses do not align with the circular trajectory they were sampled from. To estimate their actual poses, start by implementing the assembly of the Gauss-Newton system of equations $\mathbf{A}\delta\boldsymbol{\chi} = \mathbf{b}$, where you will first have to choose a variable ordering (we recommend first all the camera poses, then all the landmarks).
Note the use of `scipy.spatial.transform.Rotation` that seamlessly represents any form of rotation, i.e. can be constructed from and converted to rotation matrix, quaternion, or rotation vector.

iii) Now implement the Levenberg-Marquardt minimisation. Check the convergence in the 3D plot and implement termination criteria for the relative change in cost and absolute change in variables optimised.

iv) Finally, we have implemented a way to corrupt the data associations with outliers using an outlier probability P_{outlier}. Set it to a value higher than 0 and observe the convergence. To mitigate the effect, apply the following hard outlier robust cost function

$$\rho(r) = \begin{cases} \frac{1}{2}r^2, r \le r_{\max}, \\ \frac{1}{2}r_{\max}^2, r > r_{\max}. \end{cases}$$

Hint: this has the advantage of a relatively simple implementation, because cost and Jacobians remain unchanged for $r \le r_{\max}$, whereas the individual cost/error terms become constant for $r > r_{\max}$, and the Jacobians, respectively, becoming zero.